Aula 03 - Encapsulamento

Modificadores de Acesso, *Getters*, *Setters* e Validação de Dados

Prof. Dr. Welington Galvão Rodrigues

Análise e Desenvolvimento de Sistemas - ADS
IFMA — Campus Grajaú

22 de agosto de 2026

INSTITUTO FEDERAL
Maranhão

INSTITUTO FEDERAL
Maranhão

Agenda

1. Introdução
2. O Problema dos Atributos Públicos
3. Modificadores de Acesso
4. Getters e Setters
5. Validação de Dados em Setters
6. Estudo de Caso: ContaBancaria Encapsulada
7. Síntese



Introdução

Retomada da Aula 02

- O que é um construtor e por que ele é executado automaticamente?
- O que significa sobrecarregar um construtor?
- Para que serve a palavra-chave `this`?
- Qual a diferença entre método de instância e método estático?

Aula de reposição
Hoje repomos a quarta-feira 19/08 (Feriado: Dia do Evangélico), em sábado letivo já previsto no calendário acadêmico institucional.

Agenda de Hoje

Até agora, os atributos de nossas classes (`ContaBancaria`, `Livro`, `Aluno`...) foram acessados **diretamente**, sem nenhuma restrição.

> **O problema**
> Qualquer código externo pode alterar um atributo livremente — inclusive atribuindo valores **inválidos**.

Hoje resolvemos isso com **encapsulamento**.



O Problema dos Atributos Públicos

O Bug do Saldo Negativo

```java
public class ContaBancaria {
    public String titular;
    public double saldo;
}
```

```java
ContaBancaria c = new ContaBancaria();
c.titular = "Maria Silva";
c.saldo = -5000.0;    // saldo negativo, sem nenhum saque!
```

O Bug do Saldo Negativo (cont.)

Saída: `Saldo de Maria Silva:  R$ -5000.0`

O compilador acusou erro?
Não! Sintaticamente está tudo correto. O problema é de **design**: com `saldo` público, qualquer código pode colocar o objeto em um estado **inválido**.

O que é Encapsulamento?

Encapsulamento é o pilar da POO que consiste em **esconder os detalhes internos** de um objeto e expor apenas o necessário por meio de uma **interface controlada**.

> *“Esconda os dados, exponha o comportamento.”*
>
> — resumo clássico do encapsulamento

Analogia: a Câmera Fotográfica Automática

Ao clicar o botão da câmera, mecanismos internos selecionam velocidade e abertura do obturador conforme a iluminação — o usuário não manipula esses dados diretamente, só pede a foto.

> **Ocultação de dados**
>
> O botão é a **interface pública**; os cálculos internos são o **estado encapsulado**, protegido de manipulação direta.

Analogia adaptada de SANTOS, R. — *Introdução à POO usando Java*, cap. 1.

Por que Encapsular?

- **Integridade dos dados**: evita estados inválidos
- **Controle de acesso**: o programador decide o que é lido/alterado
- **Flexibilidade**: a implementação interna pode mudar sem quebrar quem usa a classe
- **Validação centralizada**: regras de negócio em um só lugar



Modificadores de Acesso

Os Quatro Níveis de Acesso

Modificador	Mesma classe	Mesmo pacote	Subclasse	Qualquer lugar
`private`	✓			
(padrão)	✓	✓		
`protected`	✓	✓	✓	
`public`	✓	✓	✓	✓

Baseado em FURGERI, S. — *Java 8 Ensino Didático*, seção 7.6.

MODIFICADORES DE ACESSO EM JAVA: ENCAPSULAMENTO

PRIVATE



Acesso apenas dentro da **PRÓPRIA CLASSE.**

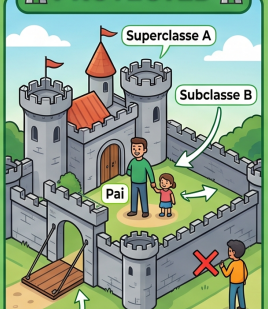
(DEFAULT)

(Sem Modificador)



Acesso dentro do **MESMO PACOTE** (Package).

PROTECTED



Acesso dentro do **MESMO PACOTE** e por **SUBCLASSES.**

PUBLIC



Acesso de **QUALQUER LUGAR** (Classes de qualquer pacote).

Objetivo: Controlar a visibilidade e o acesso aos dados para manter a integridade do objeto.

Em uma Aula sobre Encapsulamento

Convenção Prática para Encapsulamento

Regra prática de hoje

Atributos sempre `private`; métodos da interface pública sempre `public`. (`protected` será usado ao estudarmos herança.)

```java
public class ContaBancaria {
    private String titular;   // só a propria classe acessa
    private double saldo;

    // metodos publicos formam a "porta de entrada"
}
```



Getters e Setters

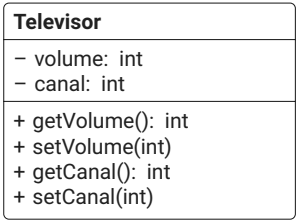
O que São Getters e Setters?

INSTITUTO FEDERAL
Maranhão

Getter
Método público que **retorna** o valor de um atributo privado. Convenção: `get` + nome do atributo (`is` + nome, se for `boolean`).

Setter
Método público que **altera** o valor de um atributo privado. Convenção: `set` + nome do atributo, retorno `void`.

UML: Classe `Televisor` Encapsulada



O sinal "**–**" indica visibilidade `private`; o sinal "**+**" indica `public`.

Baseado na Figura 7.4 de FURGERI, S. — *Java 8 Ensino Didático*.

Padrão de um Getter e de um Setter

```java
public tipo getNomeDoAtributo() {
    return nomeDoAtributo;
}

public void setNomeDoAtributo(tipo nomeDoAtributo) {
    this.nomeDoAtributo = nomeDoAtributo;
}
```

Padrão descrito em FURGERI, S. — seção 7.6 (métodos `get`/`set`).

Live Coding: `Televisor` — Atributos

```java
public class Televisor {
    private int volume;
    private int canal;
    private boolean ligado;

    public Televisor(int volume, int canal) {
        this.volume = volume;
        this.canal = canal;
        this.ligado = false;
    }
}
```

Live Coding: `Televisor` — Getters

```java
public int getVolume() {
    return volume;
}

public int getCanal() {
    return canal;
}

public boolean isLigado() {   // boolean: prefixo "is"
    return ligado;
}
```

Live Coding: `Televisor` — Setters

```java
public void setVolume(int volume) {
    this.volume = volume;
}

public void setCanal(int canal) {
    this.canal = canal;
}

public void ligar() {
    this.ligado = true;
}
```

Testando a Classe

```
1  Televisor tv = new Televisor(20, 5);
2
3  System.out.println("Volume: " + tv.getVolume());    // 20
4  tv.setVolume(35);
5  System.out.println("Novo volume: " + tv.getVolume());    // 35
6
7  // tv.volume = 999;    // ERRO: volume e' private
```



Validação de Dados em Setters

Getters/Setters Não Resolvem Tudo Sozinhos

```java
public void setSaldo(double saldo) {
    this.saldo = saldo;    // aceita QUALQUER valor, ate negativo!
}
```

Pergunta

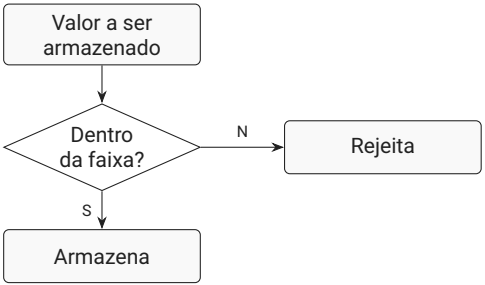
Um getter e um setter, por si só, garantem a integridade dos dados? O que está faltando?

Validação: o Setter como Guardião

O setter deixa de ser um simples "copiador de valor" e passa a ser o **guardião das regras de negócio** daquele atributo.

- **Rejeitar** o valor inválido (mantém o valor anterior);
- **Ajustar** para um padrão seguro;
- **Lançar uma exceção** (técnica mais robusta, veremos em aulas futuras).

Fluxo de Validação



Reprodução do fluxograma da Figura 7.5 de FURGERI, S. — *Java 8 Ensino Didático*, seção 7.6.

Exemplos de Regras de Negócio

Atributo	Regra de negócio
Saldo	saque permitido só até R$ 1000,00 por vez
Preço	aposentados têm 10% de desconto
Idade	valor deve estar entre 0 e 120

Tabela 7.2 de FURGERI, S. — *Java 8 Ensino Didático*, seção 7.6.

Live Coding: `Pessoa` com Setter Validado

```java
public class Pessoa {
    private String nome;
    private int idade;

    public void setIdade(int idade) {
        if (idade < 0 || idade > 130) {
            System.out.println("Idade invalida.");
            return;
        }
        this.idade = idade;
    }
}
```

Reaproveitando o Setter no Construtor

```java
public Pessoa(String nome, int idade) {
    this.nome = nome;
    setIdade(idade);   // reaproveita a validacao do setter!
}
```

Vantagem

Ao chamar `setIdade(idade)` no construtor, a validação passa a valer também na **criação** do objeto.



Estudo de Caso: ContaBancaria Encapsulada

Estado Inicial: o Problema

```java
public class ContaBancaria {
    public String titular;
    public int numero;
    public double saldo;
}
```

Atributos públicos permitem `c.saldo = -5000.0;` sem nenhuma regra de negócio.

Passo 1 — Atributos Privados

```java
public class ContaBancaria {
    private static int totalDeContas = 0;

    private String titular;
    private int numero;
    private double saldo;

    public ContaBancaria(String titular, int numero, double saldo) {
        this.titular = titular;
        this.numero = numero;
        this.saldo = saldo;
        totalDeContas++;
    }
}
```

Passo 2 — Getters e Setter Validado

```java
public String getTitular() { return titular; }
public int getNumero()     { return numero; }
public double getSaldo()   { return saldo; }

public void setSaldo(double saldo) {
    if (saldo < 0) {
        System.out.println("Saldo nao pode ser negativo.");
        return;
    }
    this.saldo = saldo;
}
```

INSTITUTO FEDERAL
Maranhão

Passo 2 — Discussão

Para a turma

Faz sentido oferecer um `setSaldo` público assim, ou seria melhor controlar o saldo só por `depositar()` e `sacar()`?

Não há resposta única: um `setSaldo` público facilita ajustes administrativos, mas abre uma porta mais ampla do que métodos com regras específicas embutidas.

Passo 3 — depositar() e sacar()

```java
public void depositar(double valor) {
    if (valor <= 0) { return; }
    saldo += valor;
}

public void sacar(double valor) {
    if (valor <= 0 || valor > saldo) {
        System.out.println("Operacao invalida.");
    } else {
        saldo -= valor;
    }
}
```

INSTITUTO FEDERAL
Maranhão

Testando a Versão Encapsulada

```java
ContaBancaria c = new ContaBancaria("Maria Silva", 1001, 500.0);

c.depositar(200.0);
c.sacar(9999.0);         // "Operacao invalida."
c.setSaldo(-100.0);      // "Saldo nao pode ser negativo."
```

INSTITUTO FEDERAL
Maranhão

Testando a Versão Encapsulada — Saída

Saída:

```
Operacao invalida.
Saldo nao pode ser negativo.
Saldo final:  R$ 700.0
```

```
1 // c.saldo = -5000.0;   // ERRO DE COMPILACAO: saldo e' private
```

Exercício em Sala

Refatorando a classe `Pessoa` (em duplas)

Retomem a classe `Pessoa` da Aula 02 e apliquem hoje:

- Todos os atributos `private`;
- Getters para todos os atributos;
- Setter validado para `idade` (0 a 130);
- Setter para `nome`, rejeitando vazio/`null`.



Síntese

Conceitos-Chave desta Aula

- Encapsulamento: esconder dados, expor comportamento
- `private`, padrão, `protected`, `public`
- Getter (`get`/`is`) e Setter (`set`)
- Validação em setters garante integridade
- Reaproveitar o setter no construtor
- Interface pública controlada

Referências

Referências desta aula

SANTOS, R. *Introdução à Programação Orientada a Objetos usando Java*. Seção 1.4 — Encapsulamento.

FURGERI, S. *Java 8 Ensino Didático: Desenvolvimento e Implementação de Aplicações*. São Paulo: Érica, 2015 — Seção 7.6, Encapsulamento (p. 110–114).

Próximas Aulas

Aula 04 — 26/08/2026 (quarta-feira)

Abstração e relacionamentos entre objetos: associação, agregação e composição, com introdução a diagramas de classes UML.

Encaminhamentos

Entregar a lista de exercícios 2 ao final da aula. Finalizar em casa a refatoração da classe `Pessoa`, se necessário.

Obrigado!

Dúvidas ou sugestões?

welington.rodrigues@ifma.edu.br



INSTITUTO
FEDERAL
Maranhão

Aula 03 - Encapsulamento

Modificadores de Acesso, *Getters*, *Setters* e Validação de Dados

Prof. Dr. Welington Galvão Rodrigues

Análise e Desenvolvimento de Sistemas - ADS
IFMA — Campus Grajaú

22 de agosto de 2026

INSTITUTO FEDERAL
Maranhão

INSTITUTO FEDERAL
Maranhão